

外部データの 取り込み、加工

医療データ科学実習

(Practice of Biomedical Data Science)

第4回 2026年5月12日

臨床統計学

emoto.ryo.5m@kyoto-u.ac.jp

Slidoで質疑応答に参加しよう

医療データ科学実習第2回

医療データ科学実習第2回
2025/04/15~2025/04/22
#2974 065

ライブインタラクシ...

Slido を切り替え

ダークモード ☐

Slido について

Slido を無料で試す

Q&A

Polls

質問を入力

人気 最近

質問投稿部分

1 個の質問

匿名
6 日前

0 投票

人の質問に投票できる
人気の質問は上位に表示される

テスト質問です

Moderator
6 日前

テスト質問了解です

- 匿名で質疑応答に参加できるプラットフォーム
- スマホからでも簡単に利用できます
- 講義後1週間解放しておくので自由に質問やコメントを投稿してください

slido

参加

URL: <https://app.sli.do/event/wp2wKuZmHwCmhceESbLsbr>



Chap0. 本日の到達目標

1. 外部データの取り込み (import) の方法を理解する
2. エクセルデータの取り込みにおける問題点とその解決法を理解する
3. データセットの形式 long format と wide format の違いを理解する
4. パッケージを用いたデータフレームの操作を知る(補足)

Chap1. 外部データの取り込み

(復習)作業ディレクトリの確認、変更

- `getwd()`: Rstudioを実行しているディレクトリ(フォルダの住所)を確認
- `setwd()`: 作業ディレクトリを変更
 - メニューから操作も可能
 1. RStudioの上部メニュー [Session] をクリック
 2. [Set Working Directory] を選択
 3. [Choose Directory...] で、保存したいフォルダを選ぶ(ない場合は作ることも可能)
 - 例: `"/Users/ryoemoto/Documents/医療データ科学実習 /第3回"`に保存したい場合

```
> # 作業ディレクトリを変更
> setwd("/Users/ryoemoto/Documents/医療データ科学実習 /第3回")
> # 確認
> getwd()
[1] "/Users/ryoemoto/Documents/医療データ科学実習 /第3回"
```

補足

例: "/Users/ryoemoto/Documents/医療データ科学実習/第3回"に保存したい場合

```
> # 作業ディレクトリを変更
> setwd("/Users/ryoemoto/Documents/医療データ科学実習 /第3回")
> # 確認
> getwd()
[1] "/Users/ryoemoto/Documents/医療データ科学実習 /第3回"
```

※先にフォルダ (Windows)やFinder(Mac)などでフォルダを作成してください

※作成したフォルダを右クリックしてプロパティ (Windows)、情報を見る (Mac)などでそのフォルダが存在するパスを確認できるのでコピーすると便利 (作成したフォルダ名を後に追加する必要あり)

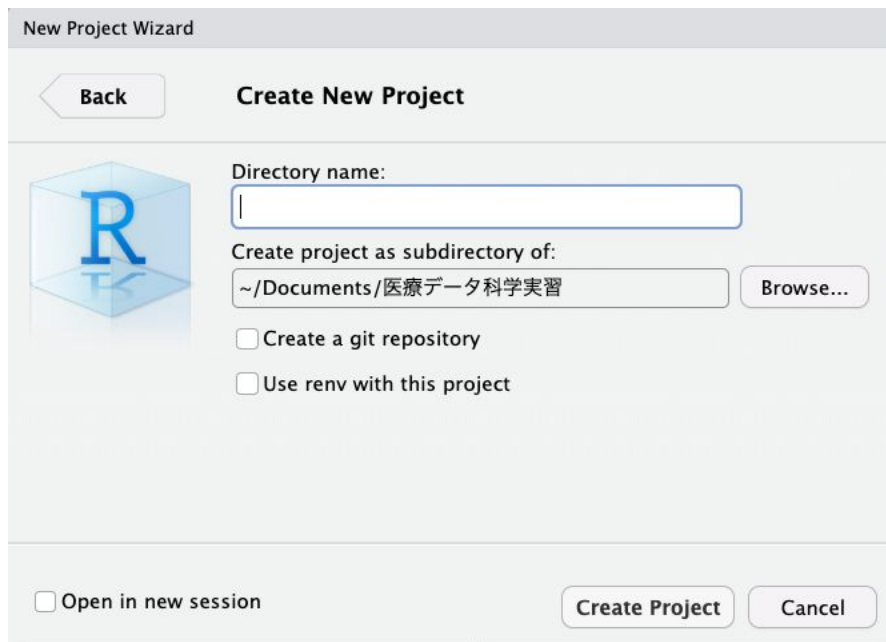
(Windowsだとパスの区切りがバックスラッシュ(\)でRで特殊文字として扱われるため \\ もしくは / に変える必要がある)

RStudio Project

- RStudio Project ファイルを作成することが可能([ファイル名].Rproj)
- フォルダ単位で解析環境を管理する仕組み
- ファイルを開くことでRStudioが起動
 - 作成したオブジェクトなどを引き継げる
 - パス指定がシンプルになる
 - `setwd()` が多くの場合必要なくなる

RStudio Projectの作成

1. RStudioの上部メニュー [File] から [New Project] を選択
2. 出てきたウィンドウで [New Directory] → [New Project] と選択
3. Directory name: に
プロジェクト名を指定
4. Create project as subdirectory of: に
プロジェクトを作成したい
フォルダを指定



フォルダ構成

- 1つの解析に1つのRStudio Project ファイルを作成
- よく用いられるフォルダ構成

project/	← プロジェクトを作成したフォルダ
└ project.Rproj	← RStudio Project ファイル
└ data/	← 解析で使用するデータを格納するフォルダ
└ script/	← 解析で使用するRコードを格納するフォルダ
└ output/	← 解析の結果を格納するフォルダ

準備1

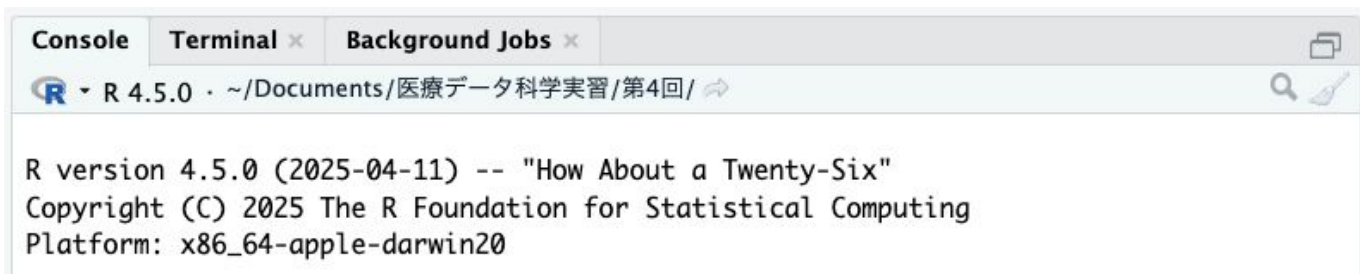
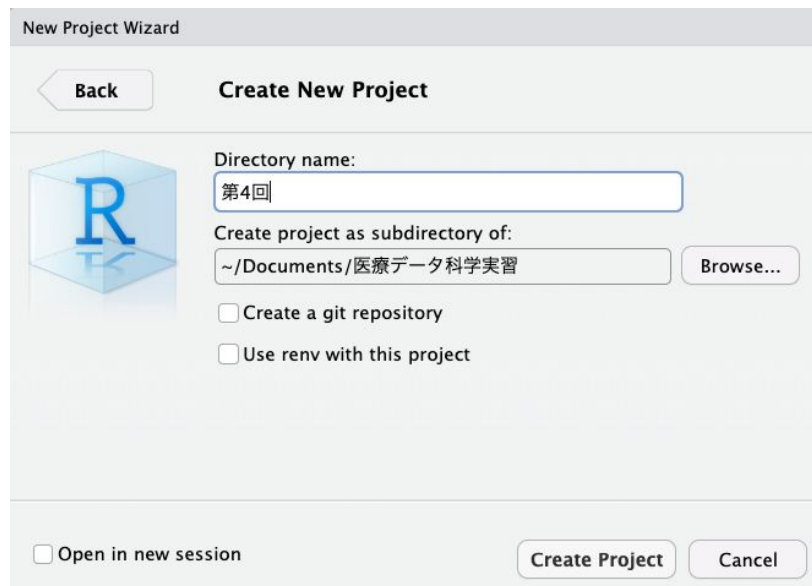
1. RStudio Project を作成

- 例:
"/Users/ryoemoto/
Documents/医療データ科学実習"
に"第4回.Rproj"を作成

2. Console画面で

現在の作業ディレクトリ

がプロジェクトの作成フォルダとなっていることを確認



準備2

3. 各自のPCに講義ページのデータを全てダウンロード
 - 上部の [ダウンロード] を選択することでzipファイルがダウンロードされる
 - ダウンロードしたzipファイルをダブルクリックしてできるフォルダ [data] を Rstudio Project を作成したフォルダにコピー
4. Rstudioの右側のFilesタブに、フォルダ [data] と作成したRstudio Projectがあることを確認



準備3

5. Rscript ファイルの保存フォルダの作成 (Windows: ファイル / Mac: Finder)
 - Rstudio Project を作成したフォルダで新しいフォルダ [script] を作成
6. Rscript ファイルの作成
 - RStudioの上部メニュー [File] をクリック
 - [New File] > [Rscript] を選択

→ Untitledというスクリプトのウィンドウが出現

7. Rscript ファイルの保存
 - RStudioの上部メニュー [File] をクリック
 - [Save as ...] を選択
 - 好きな名前をつけて保存
 - 上書き保存して使用

**データ解析においては、
元データからどのように結果を出したか
再現性が求められることがあるため
コードの保存は重要**

8. Rstudioの右側のFilesタブに、フォルダ [script] が追加されたことを確認

外部ファイルの取り込みの基本

- `read.csv()`: csvファイルをデータフレームとして読み込む関数

```
> df1 <- read.csv("data/df_infert3_sort.csv")
> head(df1)
```

	X	education	age	parity	induced	case	spontaneous
1	9	6-11yrs	21	1	0	1	1
2	67	12+ yrs	21	1	0	1	1
3	7	6-11yrs	23	1	0	1	0
4	83	12+ yrs	23	1	0	1	1
5	51	12+ yrs	24	3	1	1	2
6	19	6-11yrs	25	3	2	1	1

※`read.table()`関数: `read.csv()`の元になっている関数で
区切り文字の設定なども可能

Chap2. 外部データの取り込みにおける注意点

Excel ファイルの読み込み

- パッケージ "readxl" を使用
 パッケージのインストールとロード

```
> install.packages("readxl")  
> library(readxl)
```

- read_excel() 関数: 直接 Excel ファイルを読み込める

```
> df_example1 <- read_excel("data/example1.xlsx")  
> example1 <- as.data.frame(df_example1)
```

演習: 読み込み元のエクセルファイルと読み込み先のRオブジェクトを比較し
正しくデータを読み込めているか確認してください

問題点：読み込みの表示不整合

元データと読み込まれたデータとの比較

	A	B	C	E	F
1	日付	陽性数	陰性数	陽性率 (%)	
2	2025/4/1	18	102	15.0%	
3	2025/4/2	21	114	15.6%	
4	2025/04/03	16	112	12.5%	
5	2025/04/04	22	118	15.7%	
6	2025/04/05	19	113	14.4%	
7	4月6日	10	90	10.0%	
8					

元データ(エクセル)

> example1

	日付	陽性数	陰性数	検査数	陽性率 (%)
1	45748	18	102	120	15.00000
2	45749	21	114	135	15.55556
3	2025/04/03	16	112	128	12.50000
4	2025/04/04	22	118	140	15.71429
5	2025/04/05	19	113	132	14.39394
6	45753	10	90	100	0.10000

R上のデータフレーム

- 列の数が異なる
- 日付が正しく読み込めない
- 陽性率の値が異なる

プレーンテキスト形式の推奨

- 列の数が異なる
→ Excel 上に非表示の列が存在
- 日付が正しく読み込めない
→ シリアル値と文字列の日付が混在
- 陽性率の値が異なる
→ 0-1 の数値にパーセント表示を適用したセルと
0-100 の値にパーセント記号をつけたセルが混在

データ解析には、Excel ではなく、
書式情報を持たない csv や txt などのプレーンテキスト形式が推奨される

データ読み込みにおける文字化け

演習: ダウンロードしたデータ example1_cp932.csv を `read.csv()` 関数を用いて読み込んでください

```
> df_example1 <- read.csv("data/example1_cp932.csv")
```

文字化けが起きたときの対処

fileEncoding 引数を使用して読み込み時に文字コードを指定

```
> # 指定なしだとError
> df_example1 <- read.csv("data/example1_cp932.csv")
Error in make.names(col.names, unique = TRUE) :
  invalid multibyte string at '<93><fa><95>t'

> df_example1 <- read.csv("data/example1_cp932.csv", fileEncoding="CP932")
> df_example1
```

	日付	陽性数	陰性数	検査数	陽性率...
1	2025/4/1	18	102	120	15.00%
2	2025/4/2	21	114	135	15.60%
3	2025/4/3	16	112	128	12.50%
4	2025/4/4	22	118	140	15.70%
5	2025/4/5	19	113	132	14.40%
6	2025/4/6	10	90	100	10.00%

文字コードに関する注意点

- Mac / RStudio: UTF-8 を前提とする
- Windows: Shift-JIS (CP932) で保存されることがある

文字コードの違いにより文字化けのリスクがあるため
UTF-8 形式で保存・読み込みするのが安全

複数行ヘッダーの問題

演習: ダウンロードしたデータ example2.csv を `read.csv()` 関数を用いて読み込んでください

```
> df_example2 <- read.csv("data/example2.csv")
```

出典: 厚生労働省
「新型コロナウイルス感染症に関するオープンデータ」
(<https://www.mhlw.go.jp/stf/covid-19/open-data.html>)
をもとに加工

複数行ヘッダーの問題

上部にタイトルや単位行があり、実際の列名が2行目以降にある場合：

- read.csv では skip 引数で対応可能
- CSV 化する際に不要な行を削除しておくのも有効

```
> df_example2 <- read.csv("data/example2.csv")  
> head(df_example2[,1:4])
```

	県別新規陽性者数の推移. 日別.	X	X.1	X.2
1		2020年7月		
2	県	2020/7/1	2020/7/2	2020/7/3
3	Hokkaido	4	8	0
4	Aomori	0	0	0
5	Iwate	0	0	0
6	Miyagi	1	0	2

表のタイトルや
列の大分類が存在

```
> df_example2 <- read.csv("data/example2.csv", skip = 2)
```

先頭2行をスキップ

```
> head(df_example2[,1:4])  
      県 X2020.7.1 X2020.7.2 X2020.7.3  
1 Hokkaido      4      8      0  
2 Aomori       0      0      0  
3 Iwate        0      0      0  
4 Miyagi       1      0      2  
5 Akita        0      0      0  
6 Yamagata     0      0      0
```

Rの列名に関する制限と注意点

不適切な列名は読み込み時に自動的に修正される:

- 数字始まり: 「X」が自動付加 (1月 → X1月)
- 空白や記号: 「.」に変換 (score 1 → score.1)
- 予約語: ピリオドが付加 (if → if.)
- 重複: 番号が付く (score → score, score.1)

半角英数字+アンダースコア (_) で構成された列名が推奨

読み込みにおけるカテゴリ変数の扱い

- 医療データにはカテゴリ変数が多く含まれる
- Rでは通常これらを factor 型 で扱う
- データを取り込む時点で factor 型としてデータフレームを作成する方法

```
# 文字列の列を factor 型として読み込む
```

```
> df_example3 <- read.csv("data/example3.csv", stringsAsFactors = TRUE)
```

```
> head(df_example3)
```

	id	sex	severity	age	temperature	status
1	P001	Male	Moderate	43.0	NA	<NA>
2	P002	Female	Mild	49.9	37.2	Inpatient
3	P003	Female	Mild	29.1	37.1	Discharged
4	P004	Male	Severe	67.7	37.4	Inpatient
5	P005	Female	Mild	46.9	37.1	Inpatient
6	P006	Female	Severe	58.0	36.7	Inpatient

stringsAsFactorsを指定することで文字型の列を全てFactor型としてinput

Rにおける欠損値の扱い

- Rでは値が存在しない(欠損している)ことを NA として表示
- NA は計算に含まれると、結果も NA になる
- 各列の欠損の個数は `summary()` 関数で表示可能
- 欠損の判定には `is.na()` 関数を使用

```
> summary(df_example3)
```

	id	sex	severity	age	temperature	status
P001	: 1	Female:26	Mild :21	Min. :29.10	Min. :35.80	Discharged:14
P002	: 1	Male :24	Moderate:15	1st Qu.:45.70	1st Qu.:36.70	Inpatient :14
P003	: 1		Severe :14	Median :51.45	Median :36.90	Unknown :14
P004	: 1			Mean :51.60	Mean :36.87	NA's : 8
P005	: 1			3rd Qu.:59.52	3rd Qu.:37.20	
P006	: 1			Max. :72.80	Max. :37.60	
(Other)	:44				NA's :6	

```
>  
> # is.na() を用いて欠損がある個数を集計  
> sum(is.na(df_example3$temperature))  
[1] 6
```

欠測の個数が表示

各要素がNAかどうかを表す論理型のベクトル

参考1: 欠損値の処理

ifelse() を使用して特定の値を NA に置換する:

```
> # "Unknown" を NA に置換 (文字列として処理)
> df_example3$status <- ifelse(as.character(df_example3$status) == "Unknown",
+                               NA, as.character(df_example3$status))
> # factor 型への再変換
> df_example3$status <- factor(df_example3$status)
> summary(df_example3)
```

	id	sex	severity	age	temperature	status
P001	: 1	Female:26	Mild :21	Min. :29.10	Min. :35.80	Discharged:14
P002	: 1	Male :24	Moderate:15	1st Qu.:45.70	1st Qu.:36.70	Inpatient :14
P003	: 1		Severe :14	Median :51.45	Median :36.90	NA's :22
P004	: 1			Mean :51.60	Mean :36.87	
P005	: 1			3rd Qu.:59.52	3rd Qu.:37.20	
P006	: 1			Max. :72.80	Max. :37.60	
(Other)	:44				NA's :6	

参考1: 欠損値の処理

ifelse() を使用して特定の値を NA に置換する:

```
> # "Unknown" を NA に置換 (文字列として処理)
```

```
> df_example3$status <- ifelse(as.character(df_example3$status) == "Unknown",  
+ NA, as.character(df_example3$status))
```

```
> # factor 型への再変換
```

```
> df_example3$status <- factor(df_example3$status)
```

```
> summary(df_example3)
```

	id	sex	severity	age	temperature	status
P001	: 1	Female:26	Mild :21	Min. :29.10	Min. :35.80	Discharged:14
P002	: 1	Male :24	Moderate:15	1st Qu.:45.70	1st Qu.:36.70	Inpatient :14
P003	: 1		Severe :14	Median :51.45	Median :36.90	NA's :22
P004	: 1			Mean :51.60	Mean :36.87	
P005	: 1			3rd Qu.:59.52	3rd Qu.:37.20	
P006	: 1			Max. :72.80	Max. :37.60	
(Other)	:44				NA's :6	

factor型を文字型に変換

NAはR上では特殊な値として扱
(引用符などはつけない/NAとい
クト名も作成できない)

Unknownというカ
テゴリを欠測(
NA)として扱う

参考2: 欠損値の処理

ifelse() を使用して NA を特定の値に置換する:

```
> # 文字列の列を factor 型として読み込む
> df_example4 <- read.csv("data/example3.csv", stringsAsFactors = TRUE)
>
> df_example4$status <- ifelse(is.na(df_example4$status), "Unknown",
as.character(df_example4$status))
> df_example4$status <- factor(df_example4$status)
> summary(df_example4)
```

	id	sex	severity	age	temperature	status
P001	: 1	Female:26	Mild :21	Min. :29.10	Min. :35.80	Discharged:14
P002	: 1	Male :24	Moderate:15	1st Qu.:45.70	1st Qu.:36.70	Inpatient :14
P003	: 1		Severe :14	Median :51.45	Median :36.90	Unknown :22
P004	: 1			Mean :51.60	Mean :36.87	
P005	: 1			3rd Qu.:59.52	3rd Qu.:37.20	
P006	: 1			Max. :72.80	Max. :37.60	
(Other)	:44				NA's :6	

参考2: 欠損値の処理

ifelse() を使用して NA を特定の値に置換する:

```
> # 文字列の列を factor 型として読み込む
> df_example4 <- read.csv("data/example3.csv", stringsAsFactors = TRUE)
>
> df_example4$status <- ifelse(is.na(df_example4$status), "Unknown",
  as.character(df_example4$status))
> df_example4$status <- factor(df_example4$status)
> summary(df_example4)
```

NAかどうかをis.na()を用いて判定

"Unknown", NA ⇒ 文字列"Unknown"

NAでない ⇒ 元のデータ(文字列)

	id	sex	severity	age	temperature
P001	: 1	Female:26	Mild :21	Min. :29.10	Min. :35.80
P002	: 1	Male :24	Moderate:15	1st Qu.:45.70	1st Qu.:36.70
P003	: 1		Severe :14	Median :51.45	Median :36.90
P004	: 1			Mean :51.60	Mean :36.87
P005	: 1			3rd Qu.:59.52	3rd Qu.:37.20
P006	: 1			Max. :72.80	Max. :37.60
(Other)	:44				NA's :6

status
Discharged:14
Inpatient :14
Unknown :22

NAをUnknownという1カテゴリとして扱う

演習問題1

1. exercise1.xlsx を開き、以下の処理をして CSV で保存してください。
 - ・ヘッダーフッターの削除、列名の修正、不要列の削除
 - ・文字コード UTF-8 の CSV ファイルとして保存
2. 修正した CSV ファイルを RStudio 上で読み込んでください。
3. summary() 関数を用いてデータを確認してください。
4. class() 関数を用いてデータの各列のデータ型を確認してください。
5. 文字型として読み込まれている列(大阪の検査中の症例数)を数値型に変換してください。

演習問題1: コード例

エクセルデータの修正

	A	B	C	D	E	F	G	H	I	J
1	Date	Shiga_inpatient	Shiga_discharged	Shiga_pending	Kyoto_inpatient	Kyoto_discharged	Kyoto_pending	Osaka_inpatient	Osaka_discharged	Osaka_pending
2	2020/7/1	3	99	0	17	346	0	45	1712	-
3	2020/7/2	3	99	0	19	346	0	51	1714	-

読み込みとデータフレームの加工

```
df_ex1 <- read.csv("data/exercise1_clean.csv")
summary(df_ex1)
class(df_ex1[,10])
df_ex1[,10]

df_ex1[,10] <- as.numeric(ifelse(df_ex1[,10] == "-", NA, df_ex1[,10]))
summary(df_ex1)
class(df_ex1[,10])
df_ex1[,10]
```

Chap3. データセットの形式

Wide format (横持ち)

- 1人や1施設が1行にまとまっており、列が多くなる形式
- 入力や確認に適している

例1: 新規感染者数データ

Prefecture	2020/07/01	2020/07/02	2020/07/03
Tokyo	67	107	124
Osaka	10	8	11
Hokkaido	4	8	0

例2: バイタルサイン

Patient ID	Temp_AM	Temp_PM	BP_AM	BP_PM
P001	36.5	36.8	120	115
P002	37.0	37.2	130	125

良いデータセットとは？

- 各列が「1つの変数」である
- 各行が「1つの観測単位(レコード)」である
- 不要な装飾・合計行が含まれない
- 欠損値は NA など明記されている
- 列名は扱いやすい形(英数字＋アンダースコアなど)

データ解析しやすい構造にすることが重要

→ 整理された構造 = 処理・再利用が容易

Long format (縦持ち)

- 1人複数行のデータ
- 分析や可視化、整形に適した構造

例1: 新規感染者数

Prefecture	Date	New cases
Tokyo	2020/07/01	67
Tokyo	2020/07/01	107
Tokyo	2020/07/01	124
Osaka	2020/07/02	10
Osaka	2020/07/02	8
Osaka	2020/07/02	11
Hokkaido	2020/07/03	4
Hokkaido	2020/07/03	8
Hokkaido	2020/07/03	0

例2: バイタルサイン

Patient ID	Time	Temp	BP
P001	AM	36.5	120
P001	PM	36.8	115
P002	AM	37.0	130
P002	PM	37.2	125

wide format から long format への加工

- `reshape()` 関数の基本的な使い方: コード例

```
> df_wide <- data.frame( Patient_ID = c("P001", "P002"), Temp_AM = c(36.5, 37.0), Temp_PM = c(36.8, 37.2), BP_AM = c(120, 130), BP_PM = c(115, 125))
> df_wide
  Patient_ID Temp_AM Temp_PM BP_AM BP_PM
1      P001    36.5    36.8   120   115
2      P002    37.0    37.2   130   125
>
> df_long1 <- reshape(
+   df_wide,
+   varying = c("Temp_AM", "Temp_PM"), # varying: 縦にまとめる列を指定
+   drop = c("BP_AM", "BP_PM"), # drop: 使わない列を削除
+   v.names = "Temp",          # v.names: まとめた後の列名 (体温)
+   timevar = "Time",          # timevar: 「いつの測定か」を入れる列名
+   times = c("AM", "PM"),    # times: timevarで指定した列に入れる値
+   idvar = "Patient_ID",     # idvar: 個体識別の列 (固定)
+   direction = "long"       # direction: long formatへ変換か wide formatへ変換かの指定
+ )
> df_long1
  Patient_ID Time Temp
P001.AM      P001  AM 36.5
P002.AM      P002  AM 37.0
P001.PM      P001  PM 36.8
P002.PM      P002  PM 37.2
```

wide format から long format への加工2

- `reshape()` 関数の使い方: 二つの変数を作成する場合

```
> df_long2 <- reshape(
+   df_wide,
+   # 縦にまとめる列をリストで指定 (1つ目: 体温, 2つ目: 血圧)
+   varying = list(c("Temp_AM", "Temp_PM"), c("BP_AM", "BP_PM")),
+   v.names = c("Temp", "BP"),          # まとめた後の列名 (体温、血圧)
+   timevar = "Time",                  # 「いつの測定か」を入れる列名
+   times = c("AM", "PM"),            # timevarで指定した列に入れる値
+   idvar = "Patient_ID",              # 個体識別の列 (固定)
+   direction = "long"                # 縦持ちへ変換
+ )
> df_long2
```

	Patient_ID	Time	Temp	BP
P001.AM	P001	AM	36.5	120
P002.AM	P002	AM	37.0	130
P001.PM	P001	PM	36.8	115
P002.PM	P002	PM	37.2	125

演習問題2

1. 演習問題2で加工したデータは入院患者数の各都道府県のデータが横に並んだ wide formatになっています。reshape() 関数を用いて解析のしやすい long formatへ変換してください。

ヒント: 今回のデータは既に時系列については縦に並んでいるので
県についても縦に並べたい
reshape() 関数は時系列データを想定して作られた関数なので
引数の名前がtimeなどとなっているが
今回はそこに時間ではなく場所が入る

演習問題2: コード例

```
> str(df_ex1)
'data.frame': 31 obs. of  10 variables:
 $ Date          : chr  "2020/7/1" "2020/7/2" "2020/7/3" "2020/7/4" ...
 $ Shiga_inpatient : int   3 3 3 3 3 3 3 4 4 4 ...
 $ Shiga_discharged: int  99 99 99 99 99 99 99 99 99 99 ...
 $ Shiga_pending   : int   0 0 0 0 0 0 0 0 0 0 ...
 $ Kyoto_inpatient : int  17 19 21 21 21 35 34 42 41 40 ...
 $ Kyoto_discharged: int  346 346 347 347 347 348 352 353 356 362 ...
 $ Kyoto_pending   : int   0 0 0 0 0 0 0 0 0 0 ...
 $ Osaka_inpatient : int  45 51 51 51 76 77 77 77 79 96 ...
 $ Osaka_discharged: int 1712 1714 1714 1714 1723 1730 1742 1742 1750 1763 ...
 $ Osaka_pending   : num  NA NA NA NA NA NA NA NA NA NA ...

> col_names <- colnames(df_ex1)
> (col_inpatient <- col_names[grep("inpatient",col_names)])
[1] "Shiga_inpatient" "Kyoto_inpatient" "Osaka_inpatient"
> (col_discharged <- col_names[grep("discharged",col_names)])
[1] "Shiga_discharged" "Kyoto_discharged" "Osaka_discharged"
> (col_pending <- col_names[grep("pending",col_names)])
[1] "Shiga_pending" "Kyoto_pending" "Osaka_pending"
```

演習問題2: コード例

```
> df_ex1_long <- reshape(
+   df_ex1,
+   varying = list(col_inpatient,col_discharged,col_pending),
+   v.names = c("inpatient", "discharged", "pending"),      # まとめた後の列名
+   timevar = "Prefecture",                                # 「どこの測定か」を入れる列名
+   times    = c("Shiga", "Kyoto", "Osaka"),                # timevarで指定した列に入れる値
+   idvar     = "Date",                                     # 個体識別の列（固定）
+   direction = "long"                                     # 縦持ちへ変換
+ )
>
> str(df_ex1_long)
'data.frame': 93 obs. of  5 variables:
 $ Date      : chr  "2020/7/1" "2020/7/2" "2020/7/3" "2020/7/4" ...
 $ Prefecture: chr  "Shiga" "Shiga" "Shiga" "Shiga" ...
 $ inpatient : int   3 3 3 3 3 3 3 4 4 4 ...
 $ discharged: int   99 99 99 99 99 99 99 99 99 99 ...
 $ pending   : num   0 0 0 0 0 0 0 0 0 0 ...
```


提出課題

演習問題2で作成した long format のデータフレームを
csvファイルに出力して提出してください

提出方法: KULMSの課題提出機能

提出期限: 5/13(水) 23:55

Chap 4. 補足資料

パッケージを用いたデータセットの操作

dplyr によるデータの操作

dplyr データ操作・集計の基本ツールが含まれるパッケージ

大きな特徴:

パイプ演算子 `%>%` を使用することでデータ処理の流れを綺麗に記述できる

パイプ演算子 `%>%`: 左側のオブジェクトを右側の関数に渡す演算子

演習: RStudioで下記のコードを実行してインストール・インポートしてみよう

- コンソールで以下を実行

```
install.packages("dplyr", repos="https://cloud.r-project.org")
```

- インストールが完了したら `library(dplyr)` でインポート

dplyr の使い方

- パイプ演算子 `%>%` の使い方

例 第3回 演習問題2, 3: 列の抽出→型変換→行の抽出→ソート→先頭行の表示

```
library(dplyr)
infert %>%
  select(1:6) %>%
  mutate(
    induced = as.factor(induced),
    case = as.factor(case),
    spontaneous = as.factor(spontaneous)
  ) %>%
  filter(case == 1) %>%
  arrange(age) %>%
  head()
```

`%>%` の左のオブジェクトを右の操作に流す

パッケージなし(p, 59, 60)と比較すると比較的綺麗にコードを書ける

dplyr の使い方: 行、列の抽出

- `select()` 関数: 列の抽出

```
# 列名で指定
infert %>% select(age, case)
# 列番号で指定
infert %>% select(2, 3)
# 連続した列
infert %>% select(2:4)
```

- `pull()` 関数: 列の抽出とベクトル化

```
# 列名で指定
infert %>% pull(age)
# 列番号で指定
infert %>% pull(2)
```

- `slice()` 関数: 行の抽出

```
# 行番号で指定
infert %>% slice(2)
```

dplyr の使い方: 条件を満たす行の抽出

- `filter()` 関数:
条件は入力するデータセットに含まれる変数名を使って書ける

```
> x1 = 1:5
> x2 = c("Control", "Treatment", "Treatment", "Control", "Treatment")
> x3 = c(45, 62, 38, 55, NA)
> df2 <- data.frame(ID = x1, Group = x2, Age = x3)
>
> df2 %>% filter(Age > 60)
  ID      Group Age
1  2 Treatment  62
2  5 Treatment  71
>
> # 複数条件も簡潔に記述可能
> df2 %>% filter(Age > 60, Group == "Treatment")
  ID      Group Age
1  2 Treatment  62
2  5 Treatment  71
```

dplyr の使い方: 列の追加・変換

- `mutate()` 関数:
新しい列の作成や既存列の変換を行う

```
> df2 %>% mutate(Age_cat = ifelse(Age < 65, "<65", "≥65"))
  ID      Group Age Weight Age_cat
1  1    Control  45     60    <65
2  2 Treatment  62     75    <65
3  3 Treatment  38     52    <65
4  4    Control  55     68    <65
5  5 Treatment  71     80    ≥65
>
> # 複数列の同時作成も可能
> df2 %>% mutate(x = Age + 1, y = Age * 2)
  ID      Group Age Weight  x   y
1  1    Control  45     60  46  90
2  2 Treatment  62     75  63 124
3  3 Treatment  38     52  39  76
4  4    Control  55     68  56 110
5  5 Treatment  71     80  72 142
```

dplyr の使い方: 欠測の処理

- mutate() 関数と na_if() 関数: 欠損処理

```
> summary(df_example3)
```

	id	sex	severity	age	temperature	status
P001	: 1	Female:26	Mild :21	Min. :29.10	Min. :35.80	Discharged:14
P002	: 1	Male :24	Moderate:15	1st Qu.:45.70	1st Qu.:36.70	Inpatient :14
P003	: 1		Severe :14	Median :51.45	Median :36.90	Unknown :14
P004	: 1			Mean :51.60	Mean :36.87	NA's : 8
P005	: 1			3rd Qu.:59.52	3rd Qu.:37.20	
P006	: 1			Max. :72.80	Max. :37.60	
(Other)	:44				NA's :6	

```
> df_example3 %>% mutate(
+   status = as.character(status),
+   status = na_if(status, "Unknown"),
+   status = factor(status)
+ ) %>% summary()
```

	id	sex	severity	age	temperature	status
P001	: 1	Female:26	Mild :21	Min. :29.10	Min. :35.80	Discharged:14
P002	: 1	Male :24	Moderate:15	1st Qu.:45.70	1st Qu.:36.70	Inpatient :14
P003	: 1		Severe :14	Median :51.45	Median :36.90	NA's :22
P004	: 1			Mean :51.60	Mean :36.87	
P005	: 1			3rd Qu.:59.52	3rd Qu.:37.20	
P006	: 1			Max. :72.80	Max. :37.60	
(Other)	:44				NA's :6	

dplyr の使い方: 行のソート

- `arrange()` 関数:
指定した列に基づいて行をソート

```
> df2 %>% arrange(Age)
  ID      Group Age Weight
1  3 Treatment  38      52
2  1   Control  45      60
3  4   Control  55      68
4  2 Treatment  62      75
5  5 Treatment  71      80
>
> # 降順の場合
> df2 %>% arrange(desc(Age))
  ID      Group Age Weight
1  5 Treatment  71      80
2  2 Treatment  62      75
3  4   Control  55      68
4  1   Control  45      60
5  3 Treatment  38      52
```

dplyr の使い方: データフレームの結合

- `xxx_join()` 関数: `merge()` 関数に対応する関数で下記の種類がある
 - `inner_join()`: 共通部分のみ
 - `left_join()`: 左のデータを基準
 - `full_join()`: すべて残す

```
> df2 %>% inner_join(df_outcome2, by = "ID")
  ID   Group Age Weight Recurrence Survival_Months
1  1 Control  45    60         No             60
2  2 Treatment 62    75         Yes             24
3  3 Treatment 38    52         No             48
> df2 %>% left_join(df_outcome2, by = "ID")
  ID   Group Age Weight Recurrence Survival_Months
1  1 Control  45    60         No             60
2  2 Treatment 62    75         Yes             24
3  3 Treatment 38    52         No             48
4  4 Control  55    68        <NA>             NA
5  5 Treatment 71    80        <NA>             NA
> df2 %>% full_join(df_outcome2, by = "ID")
  ID   Group Age Weight Recurrence Survival_Months
1  1 Control  45    60         No             60
2  2 Treatment 62    75         Yes             24
3  3 Treatment 38    52         No             48
4  4 Control  55    68        <NA>             NA
5  5 Treatment 71    80        <NA>             NA
6  6    <NA>  NA    NA         No             55
7  7    <NA>  NA    NA         Yes             12
```

tidyr の使い方: データ形式の加工1

- `pivot_longer()` 関数: `reshape()` 関数よりも直感的な指定が可能

```
> #install.packages("tidyr")
> library(tidyr)
> library(dplyr)
>
> # pivot_longer() を使った Long形式への変換
> df_long_tidy1 <- df_wide %>%
+   pivot_longer(
+     cols      = c("Temp_AM", "Temp_PM"), # 縦にまとめる列
+     names_to  = "Time",                  # 列名 (AM/PM等) を入れる新しい列名
+     values_to = "Temp"                  # 数値を入れる新しい列名
+   )
> head(df_long_tidy1)
# A tibble: 4 × 5
  Patient_ID BP_AM BP_PM Time      Temp
  <chr>      <dbl> <dbl> <chr>    <dbl>
1 P001        120   115 Temp_AM  36.5
2 P001        120   115 Temp_PM  36.8
3 P002        130   125 Temp_AM   37
4 P002        130   125 Temp_PM  37.2
```

tidyr の使い方: データ形式の加工2

- `pivot_longer()` 関数: 列名が規則的な場合

```
> df_long_tidy2 <- df_wide %>%  
+   pivot_longer(  
+     cols      = -Patient_ID,          # Patient_ID "以外" の全列を対象にする  
+     names_to  = c(".value", "Time"),  # 区切った前半を「列名」に、後半を「Time列」に  
+     names_sep = "_"                  # アンダーバーで列名を分割する  
+   )  
> df_long_tidy2  
# A tibble: 4 × 4  
  Patient_ID Time    Temp    BP  
  <chr>      <chr> <dbl> <dbl>  
1 P001      AM    36.5  120  
2 P001      PM    36.8  115  
3 P002      AM    37    130  
4 P002      PM    37.2  125
```

- `pivot_wider()` 関数: 逆の加工も可能